

StaySync — BFF

Migración de Seguridad Perimetral

De autenticación propia en el BFF a AWS Cognito + AWS API Gateway

Documento técnico de arquitectura — Objetivo 1

Servicio: bff-service · Patrón objetivo: Backend Confiado (Zero Trust simplificado)

Estado: propuesta de referencia — no aplicada aún en el código en ejecución. Ver sección 5 para el motivo y los
prerrequisitos.

1. Contexto y problema actual

El servicio **bff-service** (Backend For Frontend) de StaySync está asumiendo hoy responsabilidades que corresponden a un API Gateway perimetral: valida y firma sus propios JSON Web Tokens (JWT), aplica las reglas de autorización por rol y expone endpoints de login, registro y refresco de sesión que en realidad son un simple proxy hacia **usuarios-service**.

Concretamente, el BFF concentra hoy tres piezas de lógica de seguridad:

- **JwtAuthFilter**: filtro de servlet que intercepta cada request y valida manualmente la cabecera `Authorization`.
- **JwtService**: valida la firma HMAC del token usando un *secreto compartido* (`jwt.secret`) que debe coincidir byte a byte con el que usa **usuarios-service** para firmar. Este acoplamiento por secreto compartido es frágil: cualquier desincronización rompe la autenticación de todo el sistema.
- **AuthBffController**: expone `/bff/auth/login`, `/registro`, `/refresh` y `/logout` como proxy directo a usuarios-service.

Esta mezcla de responsabilidades genera confusión arquitectónica: el BFF debería limitarse a agregar y orquestar llamadas a microservicios para el cliente web, no a emitir ni validar credenciales de identidad. El objetivo de este documento es definir con precisión dónde debe vivir cada responsabilidad de seguridad perimetral una vez adoptado AWS Cognito como proveedor de identidad (IdP/CIAM).

2. Arquitectura objetivo: Backend Confiado (Zero Trust simplificado)

La idea central del patrón es simple: **la seguridad perimetral se valida una sola vez, en el borde de la red**, y todo lo que hay detrás de ese borde (el BFF) confía en que solo puede recibir tráfico que ya fue autenticado. "Confiado" no significa "sin verificación": el BFF sigue verificando criptográficamente el token (defensa en profundidad), pero deja de ser dueño del ciclo de vida de las credenciales.

2.1 Flujo de una petición

```

Cliente Web
|
| 1. Login (Cognito Hosted UI / SDK) -> obtiene JWT (id_token / access_token)
v
AWS Cognito (User Pool)
- Fuente de verdad de identidad (IdP / CIAM)
- Emite y firma los JWT, gestiona grupos de usuario (roles)
|
| 2. Request con "Authorization: Bearer <jwt>"
v
AWS API Gateway (perímetro publico)
- JWT Authorizer contra el User Pool de Cognito
- Rechaza tokens invalidos/expirados ANTES de llegar al BFF (401 en el borde)
- Rate limiting / throttling, WAF, enrutamiento publico
- Unico componente expuesto a Internet
|
| 3. Solo trafico ya autentificado llega aqui (VPC Link / Security Group)
v
BFF (Spring Boot) -- "Backend Confiado"
- Resource Server: valida localmente firma/expiracion via JWKS de Cognito (cache, sin red)
- Mapea "cognito:groups" -> autoridades ROLE_* para @PreAuthorize
- CERO logica de emision/renovacion de credenciales
- Su unico trabajo: agregar y orquestar microservicios
|
v
Microservicios internos (usuarios, habitaciones, reservas, pagos, servicios)

```

El punto clave: la validación de firma ocurre **dos veces** por diseño, no por redundancia accidental. API Gateway la hace para proteger el perímetro público (rechazar tráfico basura lo antes posible, antes de gastar cómputo del BFF). El BFF la repite localmente contra el JWKS cacheado de Cognito, sin round-trip de red, como defensa en profundidad — así, si algún día el BFF quedara accidentalmente alcanzable sin pasar por el Gateway, seguiría rechazando tokens inválidos.

2.2 Rol de cada componente

Componente	Responsabilidad
AWS Cognito (IdP / CIAM)	Fuente única de identidad. Gestiona registro, login, MFA, recuperación de contraseña, grupos de usuario (equivalentes a los roles ADMIN / RECEPCIONISTA / HUÉSPED actuales) y emite JWT firmados (id_token, access_token, refresh_token).
AWS API Gateway (perímetro público)	Único punto expuesto a Internet. Rutas públicas vs. protegidas, JWT Authorizer contra el User Pool, rate limiting / throttling, WAF opcional. Rechaza en el borde cualquier request sin token válido — el BFF nunca llega a verla.
BFF (Spring Boot) (backend confiado)	Sin lógica de autenticación propia: sin emisión de tokens, sin secreto compartido, sin endpoints de login/registro/refresh. Usa OAuth2 Resource Server para leer claims del token ya validado en el borde, y se dedica exclusivamente a agregar/orquestar los microservicios internos.

3. Configuración de referencia en Spring Boot

En el estado objetivo, **JwtAuthFilter** y **JwtService** se eliminan por completo y se reemplazan por **spring-boot-starter-oauth2-resource-server**, el módulo estándar de Spring Security para validar JWT emitidos por un proveedor externo (Cognito) contra su JWKS público.

3.1 Dependencia Maven (pom.xml)

```
<!-- Reemplaza jjwt-api / jjwt-impl / jjwt-jackson -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
</dependency>
```

3.2 SecurityConfig

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        return http
            .csrf(AbstractHttpConfigurer::disable)
            .cors(Customizer.withDefaults())
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/swagger-ui/**", "/api-docs/**", "/actuator/health",
                    "/bff/pagos/stripe/**") // webhook: server-to-server, sin Cognito
                .permitAll()
                .anyRequest().authenticated())
            .oauth2ResourceServer(oauth2 -> oauth2
                .jwt(jwt -> jwt.jwtAuthenticationConverter(cognitoAuthConverter()))))
            .build();
    }

    @Bean
    public JwtAuthenticationConverter cognitoAuthConverter() {
        var grantedAuthoritiesConverter = new JwtGrantedAuthoritiesConverter();
        grantedAuthoritiesConverter.setAuthoritiesClaimName("cognito:groups");
        grantedAuthoritiesConverter.setAuthorityPrefix("ROLE_");

        var converter = new JwtAuthenticationConverter();
        converter.setJwtGrantedAuthoritiesConverter(grantedAuthoritiesConverter);
        return converter;
    }
}
```

Los grupos de Cognito (cognito:groups) reemplazan la lógica actual de **JwtAuthFilter** que arma manualmente un **SimpleGrantedAuthority("ROLE_" + rol)** a partir del claim propio **rol**. Las anotaciones **@PreAuthorize** existentes en los controladores no cambian.

3.3 application.yml

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: ${COGNITO_ISSUER_URI:https://cognito-idp.<region>.amazonaws.com/<user-pool-id>}
```

Spring resuelve automáticamente el endpoint JWKS a partir del *issuer-uri* y cachea las claves públicas de Cognito en memoria: la validación de firma en el BFF no implica una llamada de red por cada request.

3.4 Qué se elimina del BFF

- `JwtAuthFilter` — filtro manual de validación de cabecera Authorization.
- `JwtService` — parsing y verificación HMAC con secreto compartido.
- El secreto `jwt.secret` replicado entre `bff-service` y `usuarios-service`.
- Los endpoints de `AuthBffController` (`/login`, `/registro`, `/refresh`) — el login pasa a resolverse contra Cognito (Hosted UI o llamada directa `InitiateAuth` desde el frontend), no a través del BFF.

4. Separación de enrutamiento público vs. interno

Una consecuencia directa de este rediseño es que existen ahora **dos tablas de enrutamiento distintas, en dos capas distintas**, y no deben mezclarse:

- **Enrutamiento perimetral / público:** qué rutas exige un token de Cognito, cuáles son públicas (p. ej. el webhook de Stripe) y el rate limiting. Esto se configura en **AWS API Gateway**, no en el código del BFF.
- **Enrutamiento interno BFF → microservicios:** a qué URL interna llega cada cliente REST (`UsuariosClient`, `HabitacionesClient`, etc.). Esto sigue viviendo en `application.yml` del BFF, externalizado vía variables de entorno (`services.usuarios.url`, etc.) y ahora tipado mediante una clase `@ConfigurationProperties` (`ServicesProperties`) en lugar de `@Value` sueltos por cliente.

5. Por qué esta migración no está aplicada todavía en el código

■ Prerrequisito bloqueante

Hoy, **usuarios-service** sigue emitiendo sus propios JWT firmados con el secreto compartido. Si se reconfigura `SecurityConfig` del BFF para esperar tokens de Cognito, **todo login actual dejaría de funcionar**: ningún token emitido por `usuarios-service` sería válido contra el *issuer-uri* de Cognito.

Este repositorio no tiene control de versiones (no es un repositorio git), por lo que un cambio de este tipo no se podría revertir automáticamente si algo falla.

Por eso este documento se entrega como **referencia de diseño**, lista para aplicar cuando se cumplan los prerrequisitos, en vez de modificarse directamente sobre el código en ejecución.

6. Plan de migración recomendado

1. Aprovisionar Cognito

Crear el User Pool, definir los grupos ADMIN / RECEPCIONISTA / HUÉSPED (equivalentes al enum actual `Usuario.Rol`) y, si hay usuarios existentes en la base de usuarios-service, definir una estrategia de migración (Lambda trigger de migración de usuarios de Cognito, o registro forzado en el próximo login).

2. Migrar la emisión de tokens

usuarios-service deja de firmar JWT propios. El frontend se autentica directamente contra Cognito (Hosted UI o SDK) y obtiene el JWT de ahí.

3. Configurar AWS API Gateway

Crear el JWT Authorizer contra el User Pool, definir las rutas públicas vs. protegidas y las políticas de throttling.

4. Aplicar el nuevo SecurityConfig en el BFF

Sustituir `JwtAuthFilter/JwtService` por `spring-boot-starter-oauth2-resource-server`, como se detalla en la sección 3, una vez que el paso 2 esté en producción.

5. Retirar AuthBffController

Eliminar los endpoints de proxy de login/registro/refresh una vez que el frontend hable directamente con Cognito.

Documento generado como material de referencia técnica para el equipo de StaySync. No reemplaza una revisión de seguridad formal antes de pasar a producción.